Week No. 1

Lab Manual: Information Security

Lab Title: Classical Ciphers (Code)

Lab Objectives

By the end of this lab, students will be able to:

1. Implement and analyze classical encryption techniques through coding practice.
2. Understand the working principles of the substitution-based ciphers.
3. Evaluate strengths and weaknesses of each classical cipher in modern context.

Learning Outcomes (CLOs & PLOs Mapping)

CLO	CLO Statement	Bloom's Level	PLO
1	Apply Information Security techniques into practice.	P3 (Apply)	PLO-5

Lab Overview

This lab focuses on Classical Cryptographic Algorithms, the foundation of modern cryptosystems. Students will study and implement:

- Caesar Cipher
- Vigenère Cipher
- Auto-Key Cipher
- Playfair Cipher

How each experiment is organised: (1) Theory and a fully worked example of encryption and decryption, (2) Practice questions to be solved on paper, (3) Pseudo code, (4) The coding task.

Before You Start: Basic Terms

Term	Meaning
Plaintext	The normal, readable message (e.g. HELLO).
Ciphertext	The scrambled, unreadable message.
Encryption	Turning plaintext into ciphertext.
Decryption	Turning ciphertext back into plaintext.
Key	The secret number or word that controls the scrambling.
Cipher	The method (recipe) used to scramble the message.

The Letter-Number Table

Most classical ciphers do their work with numbers. Give every letter a number, starting from 0:

A	B	C	D	E	F	G	H	I	J	K	L	M
0	1	2	3	4	5	6	7	8	9	10	11	12

N	O	P	Q	R	S	T	U	V	W	X	Y	Z
13	14	15	16	17	18	19	20	21	22	23	24	25

The wrap-around rule (mod 26)

The alphabet has only 26 letters, so it works like a clock that loops: after Z comes A again.

If a result is **bigger than 25**, subtract 26. If a result is **below 0**, add 26.

Example: 30 − 26 = 4 (E). −7 + 26 = 19 (T).

Experiment 1: Caesar Cipher

Theory

A substitution cipher where each letter in the plaintext is shifted by a fixed number (the key) within the alphabet. Every letter moves the same number of steps.

Encryption Formula	Decryption Formula
C = (P + K) mod 26	P = (C − K) mod 26

P = plaintext letter number, C = ciphertext letter number, K = key (shift).

Worked Example: HELLO with Key = 3

Encryption

1. Convert each letter to its number.
2. Add the key (3) to each number.
3. If the result is over 25, subtract 26 (not needed here).
4. Convert the numbers back to letters.

	H	E	L	L	O
Plain number (P)	7	4	11	11	14
P + 3	10	7	14	14	17
After mod 26	10	7	14	14	17
Cipher letter	K	H	O	O	R

Ciphertext = KHOOR

Decryption

Do the opposite: subtract the key from each number (add 26 if the answer is below 0).

	K	H	O	O	R
Cipher number (C)	10	7	14	14	17
C − 3	7	4	11	11	14
Plain letter	H	E	L	L	O

Plaintext recovered = HELLO

Weakness: only 25 possible keys, so an attacker can simply try them all. Also the same letter always gives the same result (both L's became O).

Practice Questions (Solve on Paper)

Q1. Encrypt **ATTACK** using Caesar cipher with key = 5.

Answer:

Q2. Decrypt **WKLVLVIXQ** using Caesar cipher with key = 3.

Answer:

Q3. Encrypt **XYZ** with key = 4. Show clearly where you apply the wrap-around rule.

Answer:

Q4. How many different keys are possible in a Caesar cipher? Explain why this makes it easy to break.

Answer:

Answer:

Pseudo Code

Encryption

```
INPUT: Plaintext, Key
Ciphertext = empty
FOR each character ch in Plaintext:
    IF ch is an alphabet:
        P = number of ch (A=0 ... Z=25)
        C = (P + Key) mod 26      // wrap around
        Append letter of C to Ciphertext
    ELSE:
        Append ch unchanged       // spaces, digits, etc.
OUTPUT: Ciphertext
```

Decryption

```
INPUT: Ciphertext, Key
Plaintext = empty
FOR each character ch in Ciphertext:
    IF ch is an alphabet:
        C = number of ch
        P = (C - Key) mod 26      // mod keeps it 0..25
        Append letter of P to Plaintext
    ELSE:
        Append ch unchanged
OUTPUT: Plaintext
```

Task

- Implement encryption and decryption functions.
- Test with key = 3 on plaintext student's full name.
- Values must not be hardcoded.

Experiment 2: Vigenère Cipher

Theory

A polyalphabetic cipher using a keyword to vary the shift amount per letter. Caesar uses one shift for the whole message; Vigenère uses a **keyword**, and each keyword letter gives a **different shift** to the matching message letter. The keyword is **repeated** until it is as long as the message.

Encryption Formula	Decryption Formula
$C_i = (P_i + K_i) \bmod 26$	$P_i = (C_i - K_i) \bmod 26$

P_i, C_i and K_i are the numbers of the i-th plaintext, ciphertext and key letters.

Worked Example: HELLO with Keyword = KEY

Encryption

1. Repeat the keyword to match the message length.
2. Convert message letters and key letters to numbers.
3. Add them one pair at a time; subtract 26 if the sum is over 25.
4. Convert back to letters.

```
Message:  H E L L O
Key:      K E Y K E     (KEY repeats: K E Y, then K E again)
```

	H	E	L	L	O
Plain number (P)	7	4	11	11	14
Key letter	K	E	Y	K	E
Key number (K)	10	4	24	10	4
P + K	17	8	35	21	18
After mod 26	17	8	9 (35−26)	21	18
Cipher letter	R	I	J	V	S

Ciphertext = RIJVS

Notice the two L's became different letters (J and V). This is the big improvement over Caesar.

Decryption

Use the same repeated key and subtract. If the answer is below 0, add 26.

	R	I	J	V	S
Cipher number (C)	17	8	9	21	18
Key number (K)	10	4	24	10	4
C − K	7	4	−15	11	14
After adding 26 if negative	7	4	11 (−15+26)	11	14
Plain letter	H	E	L	L	O

Plaintext recovered = HELLO

Weakness: the key repeats. If an attacker finds the key length, the cipher breaks into several simple Caesar ciphers (Kasiski examination).

Practice Questions (Solve on Paper)

Q1. Encrypt **ATTACK** using keyword **KEY**.

Answer:

Q2. Encrypt **SUNRISE** using keyword **CAT**. (Write the repeated key first.)

Answer:

Q3. Decrypt **UUGTILG** using keyword **CAT**.

Answer:

Q4. In the example above, why did the two L's in HELLO produce different ciphertext letters? Would a Caesar cipher do the same?

Answer:

Answer:

Pseudo Code

Encryption

```
INPUT: Plaintext, Keyword
EXTEND Keyword by repeating it until len(Keyword) = len(Plaintext)
Ciphertext = empty
FOR i = 0 to len(Plaintext) - 1:
    P = number of Plaintext[i]   // A=0 ... Z=25
    K = number of Keyword[i]
    C = (P + K) mod 26
    Append letter of C to Ciphertext
OUTPUT: Ciphertext
```

Decryption

```
INPUT: Ciphertext, Keyword
EXTEND Keyword by repeating it until len(Keyword) = len(Ciphertext)
Plaintext = empty
FOR i = 0 to len(Ciphertext) - 1:
    C = number of Ciphertext[i]
    K = number of Keyword[i]
    P = (C - K) mod 26
    Append letter of P to Plaintext
OUTPUT: Plaintext
```

Task

- Implement encryption and decryption functions.
- Test with keyword “KEY” on plaintext “CRYPTOGRAPHY”.
- Values must not be hardcoded.

Experiment 3: Auto-Key Cipher

Theory

A modification of Vigenère Cipher where the plaintext itself becomes part of the key after the initial keyword. The starting keyword (called the **primer**) is used only once. After that, the message letters take over as the key, so the key **never repeats**.

The math is the same as Vigenère: **C = (P + K) mod 26** and **P = (C − K) mod 26**. Only the way the key is built is different.

Worked Example: HELLO with Primer = KEY

Encryption

1. Build the key: primer first, then the message letters. Key = KEY + HE = KEYHE.
2. Convert to numbers and add, exactly like Vigenère.
3. Subtract 26 if the sum is over 25; convert back to letters.

```
Message:  H E L L O
Key:      K E Y H E     (primer KEY, then message letters H, E)
```

	H	E	L	L	O
Plain number (P)	7	4	11	11	14
Key letter	K	E	Y	H	E
Key number (K)	10	4	24	7	4
P + K	17	8	35	18	18
After mod 26	17	8	9 (35−26)	18	18
Cipher letter	R	I	J	S	S

Ciphertext = RIJSS

Decryption

The full key cannot be written in advance. It is **built while decrypting**: every plaintext letter you recover is added to the key for later letters.

Pos	Cipher	Key letter used	Key came from	C − K	Plain
1	R (17)	K (10)	primer	17−10 = 7	H
2	I (8)	E (4)	primer	8−4 = 4	E
3	J (9)	Y (24)	primer	9−24 = −15 → 11	L
4	S (18)	H (7)	answer of pos 1	18−7 = 11	L
5	S (18)	E (4)	answer of pos 2	18−4 = 14	O

Plaintext recovered = HELLO

Weakness: the key is real message text, so attackers can guess common words. One wrong letter during decryption spoils every letter after it (error propagation).

Practice Questions (Solve on Paper)

Q1. Encrypt **SECRET** using primer **KEY**. (Write the full key stream first.)

Answer:

Q2. Decrypt **FEGHRIIQ** using primer **CAB**. (Build the key as you go; show a table like the one above.)

Answer:

Q3. What happens in Auto-Key decryption if you make one mistake in the 3rd letter? Why is this not a problem in Vigenère?

Answer:

Answer:

Pseudo Code

Encryption

```
INPUT: Plaintext, Initial_Key
extended_key = Initial_Key + first (len(Plaintext) - len(Initial_Key))
               letters of Plaintext
Ciphertext = empty
FOR i = 0 to len(Plaintext) - 1:
    P = number of Plaintext[i]
    K = number of extended_key[i]
    C = (P + K) mod 26
    Append letter of C to Ciphertext
OUTPUT: Ciphertext
```

Decryption

```
INPUT: Ciphertext, Initial_Key
key = Initial_Key                // key grows while decrypting
Plaintext = empty
FOR i = 0 to len(Ciphertext) - 1:
    C = number of Ciphertext[i]
    K = number of key[i]
    P = (C - K) mod 26
    Append letter of P to Plaintext
    Append letter of P to key     // recovered letter extends the key
OUTPUT: Plaintext
```

Task

- Implement encryption and decryption functions.
- Test with keyword “KEY” on plaintext “CRYPTOGRAPHY”.
- Values must not be hardcoded.

Experiment 4: Playfair Cipher

Theory

A digraph substitution cipher using a 5×5 matrix constructed from a keyword. Instead of one letter at a time, it encrypts **pairs of letters**, which hides single-letter frequency patterns.

Step A: Build the 5×5 matrix

1. Write the keyword letters, skipping any repeated letter.
2. Fill the remaining boxes with the rest of the alphabet in order, skipping letters already used.
3. A 5×5 grid has only 25 boxes, so **I and J share one box** (treat J as I).

Keyword = **MONARCHY**:

	Col 1	Col 2	Col 3	Col 4	Col 5
Row 1	M	O	N	A	R
Row 2	C	H	Y	B	D
Row 3	E	F	G	I/J	K
Row 4	L	P	Q	S	T
Row 5	U	V	W	X	Z

Step B: Split the plaintext into pairs (digraphs)

1. Remove spaces and replace J with I.
2. Cut the message into pairs of two letters.
3. If a pair has two identical letters (like LL), put an **X** between them and re-pair.
4. If one letter is left at the end, add an **X**.

Step C: Apply the three rules to each pair

Where the two letters are	Encrypt	Decrypt
Same row	Replace each with the letter to its right (wrap to start of row)	Replace each with the letter to its left
Same column	Replace each with the letter below (wrap to top)	Replace each with the letter above
Different row and column (rectangle)	Each letter stays in its own row and takes the column of the other letter	Same as encryption

Worked Example: HELLO with Keyword = MONARCHY

Encryption

Step 1: Make pairs. The first pair is HE. The next two letters are LL (identical), so put an X between them to make LX. The second L is then paired with O to make LO: **HE · LX · LO**.

Step 2: Encrypt each pair. All three pairs are rectangles. Use the grid above.

Pair	Position of letters (row, col)	Rule	Working	Result
HE	H (2,2), E (3,1)	Rectangle	H stays in row 2, goes to E's column 1 → C. E stays in row 3, goes to H's column 2 → F	CF

Pair	Position of letters (row, col)	Rule	Working	Result
LX	L (4,1), X (5,4)	Rectangle	L stays in row 4, goes to X's column 4 → S. X stays in row 5, goes to L's column 1 → U	SU
LO	L (4,1), O (1,2)	Rectangle	L stays in row 4, goes to O's column 2 → P. O stays in row 1, goes to L's column 1 → M	PM

Ciphertext = CFSUPM

How to see the rectangle (pair HE)

Draw a rectangle with H and E at opposite corners. The other two corners are C and F.

H is in row 2 → take the other corner in row 2 = **C**. E is in row 3 → take the other corner in row 3 = **F**.

Trick: each letter keeps its own row and jumps to the other letter's column.

Decryption

Split the ciphertext into pairs: **CF · SU · PM**. Use the same grid and the reverse rules (left / up; rectangle is the same).

Pair	Positions	Rule	Working	Result
CF	C (2,1), F (3,2)	Rectangle	C stays in row 2, goes to F's column 2 → H. F stays in row 3, goes to C's column 1 → E	HE
SU	S (4,4), U (5,1)	Rectangle	S stays in row 4, goes to U's column 1 → L. U stays in row 5, goes to S's column 4 → X	LX
PM	P (4,2), M (1,1)	Rectangle	P stays in row 4, goes to M's column 1 → L. M stays in row 1, goes to P's column 2 → O	LO

Result = HE LX LO → **HELXLO**. Remove the padding X by reading the meaning: **HELLO**.

Weakness: common letter pairs (like TH, HE, IN) still leave patterns, and the padding X and I/J merge can cause small ambiguities.

Practice Questions (Solve on Paper)

Q1. Build the 5×5 Playfair matrix for the keyword **SECURITY** (merge I/J). Draw the grid.

Answer:

Answer:

Q2. Split the plaintext **BALLOON** into Playfair pairs, adding X where needed.

Answer:

Q3. Using the **MONARCHY** matrix, encrypt **HOMEMO**. For each pair, write which rule you used (same row, same column or rectangle).

Answer:

Answer:

Q4. Using the **MONARCHY** matrix, decrypt **CFSUPM**. Remove the padding X in your final answer.

Answer:

Q5. Why can a Playfair pair never contain two identical letters? What is done to fix this?

Answer:

Pseudo Code

Encryption

```
INPUT: Keyword, Plaintext
// 1. Build the 5x5 matrix
letters = Keyword + ABCDEFGHIKLMNOPQRSTUVWXYZ   // J is merged with I
REMOVE duplicate letters from letters (keep first occurrence)
matrix = fill 5x5 grid row by row with letters
// 2. Prepare the plaintext
Plaintext = uppercase, letters only, J replaced by I
pairs = empty list;  i = 0
WHILE i < len(Plaintext):
    a = Plaintext[i]
    b = Plaintext[i+1] if it exists, ELSE 'X'
    IF a == b:
        add (a, 'X') to pairs;  i = i + 1
    ELSE:
        add (a, b) to pairs;    i = i + 2
// 3. Encrypt each pair
FOR each pair (a, b):
    find (row1, col1) of a and (row2, col2) of b in matrix
    IF row1 == row2:                // same row
        a' = matrix[row1][(col1 + 1) mod 5]
        b' = matrix[row2][(col2 + 1) mod 5]
    ELSE IF col1 == col2:           // same column
        a' = matrix[(row1 + 1) mod 5][col1]
        b' = matrix[(row2 + 1) mod 5][col2]
    ELSE:                           // rectangle
        a' = matrix[row1][col2]
        b' = matrix[row2][col1]
    Append a', b' to Ciphertext
OUTPUT: Ciphertext
```

Decryption

```
INPUT: Keyword, Ciphertext
BUILD matrix exactly as in encryption
SPLIT Ciphertext into pairs of 2 letters
FOR each pair (a, b):
    find (row1, col1) of a and (row2, col2) of b in matrix
    IF row1 == row2:                // same row: move LEFT
        a' = matrix[row1][(col1 - 1) mod 5]
        b' = matrix[row2][(col2 - 1) mod 5]
    ELSE IF col1 == col2:           // same column: move UP
        a' = matrix[(row1 - 1) mod 5][col1]
        b' = matrix[(row2 - 1) mod 5][col2]
    ELSE:                           // rectangle (same as encryption)
        a' = matrix[row1][col2]
        b' = matrix[row2][col1]
    Append a', b' to Plaintext
REMOVE padding X letters (by reading the meaning)
OUTPUT: Plaintext
```

Task

- Implement encryption and decryption functions.
- Test with keyword “MONARCHY” on plaintext “CRYPTOGRAPHY”.
- Values must not be hardcoded.

Class Participation Task (20 Marks)

You are required to design a hybrid encryption process using two sequential approaches.

Approach 1: Layered Classical Encryption

1. Take any keyword (e.g. “SECURE”).
2. Encrypt this key using Caesar Cipher with a shift of 3. Example: “SECURE” → Caesar(+3) → “VHFXUH”.
3. Use the resulting Caesar-encrypted key as the Playfair cipher key.
4. Encrypt your chosen plaintext (e.g. your name) using the Playfair Cipher matrix generated from this new key.
5. Observe how the first cipher (Caesar) influences the behavior of the second.

Approach 2: ASCII-Based Hybrid Cipher

1. Convert all characters (both plaintext and key) to their ASCII values.
2. Perform Caesar-like shifting using numerical operations on ASCII codes: C = (P + K) mod 128.
3. Now use the resulting ASCII sequence as input for the Playfair Cipher, but replace alphabetic mapping with ASCII range (0–127).

Discuss

- How does using ASCII instead of alphabets affect cipher strength?
- What new challenges arise in decryption?